



Powering Soft Querying in J-CO-QL with JavaScript Functions

Paolo Fosci and Giuseppe Psaila^(✉)

University of Bergamo, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci,giuseppe.psaila}@unibg.it
<http://www.unibg.it>

Abstract. The availability of large data sets represented as *JSON* documents is demanding for powerful tools able to integrate and query them. The *J-CO* Framework is a research prototype under development at University of Bergamo (Italy), specifically devised to provide practical tools to manage possibly large *JSON* data sets. Its query language, named *J-CO-QL*, provides constructs to evaluate the belonging of documents to fuzzy sets, so as to perform soft queries on *JSON* documents. In this paper, we present a real case study that shows how a novel construct, which enables defining JavaScript functions within *J-CO-QL* queries, provides users with an even improved possibility to personalize soft operators used to evaluate membership degrees.

Keywords: Soft querying on JSON data sets · Defining JavaScript functions · Fuzzy operators powered by JavaScript functions

1 Introduction

JSON (JavaScript Object Notation) [8] has become the *de-facto* standard format to represent data sets over the Internet. In fact, APIs (Application Programming Interfaces) of social media and Open Data portals [9] very often adopt this format to provide data sets. The reason is its simplicity and flexibility, because a *JSON* document is a serialized representation of main-memory objects managed by object-oriented programming languages. As a consequence, analysts need novel powerful tools to integrate and query *JSON* data sets possibly obtained from Internet sources.

At University of Bergamo (Italy), we have developed the *J-CO* Framework [15]. It is a research project whose goal is the development of prototypical tools to gather, store, manage, integrate, transform and query *JSON* data sets. The core of the framework is its query language, named *J-CO-QL*: originally [2,3], it was designed to cope with geo-tagging and heterogeneity of documents within the same data set; now, we are extending it to introduce capabilities of soft querying *JSON* documents. In this regard, *J-CO-QL* provides an instruction called **CREATE FUZZY OPERATOR**, which enables users to define fuzzy operators with complex semantics and complex membership functions [10,16].

Often complex computations, based on procedural algorithms, are necessary to actually cope with real situations. For example, to evaluate the *Close* relationship between two generic points on the Earth surface, it is necessary to evaluate the geodetic distance among them. Computing the geodetic distance is an easy task for a procedural function (it is possible to find many versions on the Web, coded in a multitude of programming languages): it would be nice to provide analysts with the capability to exploit these functions when necessary in the analysis process.

With respect to our previous works on the *J-CO* Framework and on the fuzzy extension that we are defining for the *J-CO-QL* language, in this paper we introduce also the `CREATE FUZZY JAVASCRIPT FUNCTION` instruction. This is a novel instruction that we recently added to the language; by means of it, it is possible to define JavaScript functions that can be called within expressions in any *J-CO-QL* instruction, as in the `CREATE FUZZY OPERATOR` instruction. The main contribution of the paper is to show how this novel feature significantly pushes towards the flexibility of fuzzy constructs, allowing users (analysts) to express complex semantics for operators used to evaluate membership degrees of *JSON* documents to fuzzy sets. We rely on a real-life case study to present the key instructions of the *J-CO-QL* language, so as to highlight novel features and their impact on soft querying *JSON* data sets retrieved from Internet sources.

The paper is organized as follows. Section 2 briefly introduces the background of this work, i.e., basic concepts on fuzzy sets (Sect. 2.1), the relevant literature on soft querying relational DBs (Sect. 2.2) and a short introduction to the *J-CO* Framework (Sect. 2.3). Section 3 introduces the case study and presents the contribution of the paper: Sect. 3.1 presents the novel instruction for defining JavaScript functions and the instruction for defining fuzzy operators; Sect. 3.2 shows the *J-CO-QL* instructions to retrieve and integrate data; Sect. 3.3 shows how soft querying *JSON* documents. Section 4 draws the conclusions.

2 Background

In this section, we briefly present the background of our work, i.e., basic notions on fuzzy sets (Sect. 2.1), the literature on soft querying relational databases (Sect. 2.2) and a short introduction to the *J-CO* Framework (Sect. 2.3).

2.1 Brief Introduction to Fuzzy Sets

Fuzzy Set Theory was introduced by Zadeh [17]; here, we report some basic concepts.

Let us denote by X a non-empty universe, either finite or infinite.

Definition 1. A fuzzy set $A \subseteq X$ is a mapping $A : X \rightarrow [0, 1]$. The value $A(x)$ is referred to as the *membership degree* of the element x to the fuzzy set A .

Therefore, a fuzzy set A in X is characterized by a membership function $A(x)$ that associates each element $x \in X$ with a real number in the interval $[0, 1]$; in other words, given x , the value $A(x)$ is the degree of membership of x to A .

A fuzzy set is empty if and only if its membership function is identically zero for each $x \in X$. Two fuzzy sets A and B are equal, denoted as $A = B$, if and only if $A(x) = B(x)$ for all $x \in X$. If $A(x) = 1$, x fully belongs to A . If $A(x) = 0$, x does not belong at all to A . An intermediate value $0 < A(x) < 1$ means that x belongs to A in a partial way (i.e., $A(x) = 0.9$ means that x does not completely belong to A). For example, if A stands for “young people” and x_1 and x_2 are two persons, the fact that $A(x_1) = 0.9$ and $A(x_2) = 0.3$ denotes that neither x_1 nor x_2 are completely young, but certainly x_1 is younger than x_2 . Consequently, fuzzy sets express vague or imprecise concepts on real-world entities, as well as they express vague or imprecise relationships among real-world entities.

In this paper, X is the universe set of *JSON* documents. Thus, given a document $d \in X$, the membership degree $A(d)$ denotes the extent with which d belongs to the A fuzzy set. However, a document d represents either a real-world entity or a relationship among real-world entities; so, the name of a fuzzy set represents, in a linguistic way, either a category of real-world entities or a category of relationships among real-world entities.

2.2 Related Work

Many researchers have addressed the problem of performing flexible queries on data sets and databases, by allowing users to express vague or imprecise concepts in the query. To this end, fuzzy sets and fuzzy logic have played an important role. In fact, fuzzy sets can be used to represent vague concepts, so as to specify soft selection conditions [1]: the idea is to quantify the closeness of the information carried by the proposition in relation to the considered reality.

There are two main ways to apply Fuzzy Set Theory in the context of database management systems. The first way consists in developing a fuzzy database model to manage imprecise relational data [6, 13]. However, this approach is difficult to introduce, because already-existing databases cannot be exploited as they are; furthermore, performing queries on a fuzzy relational model is, in general, highly inefficient [12].

The second way develops a fuzzy querying interface on top of a conventional relational database: the advantage of this approach is evident in our SoftSQL proposal [4], as well as in other proposals like FQUERY [11] and SQLf [5, 7]: soft-querying capabilities are made available on already existing classical relational databases. However, in the current world of *JSON* data sets, the classical relational model is no longer suitable. Another issue of these proposals is that they cannot deal with more than one membership degree for each tuple; in contrast, it is possible to guess that the same tuple (or the same document, in the *JSON* world) might belong to many fuzzy sets at the same time. These issues constitute the gap we want to bridge through *J-CO-QL*.

The fuzzy extension we are defining for the *J-CO-QL* language moves from our previous works on SoftSQL; in particular some ideas, as providing practical instructions to define customized fuzzy operators, have been kept. In [10], we showed the capabilities of *J-CO-QL* to manage the belonging of a single *JSON* document to many fuzzy sets at the same time; in this paper, the focus is towards

the introduction of JavaScript functions to empower *J-CO-QL* with procedural capabilities. At the best of our knowledge, *J-CO-QL* is the first language for querying *JSON* data sets that provides fuzzy constructs for performing soft querying; consequently, we have no counterpart in this sense.

2.3 The *J-CO* Framework

The *J-CO* Framework is the result of a research work [2,3,14,15] conducted at University of Bergamo, towards the definition of a tool able to retrieve, integrate and query possibly large collections of *JSON* documents either stored in *JSON* document stores or directly provided by Web sources.

The core of the framework is *J-CO-QL*, a novel query language specifically designed to query heterogeneous *JSON* data sets, by natively supporting geo-spatial aggregations of geo-tagged documents. In [16], we started to extend the language in order to support fuzzy sets and soft querying. In Sect. 3, the reader will see a non-trivial *J-CO-QL* script. Here, we briefly summarize the main features of the language

- *Data Model.* *J-CO-QL* works on standard *JSON* documents; however, root-level fields whose name begins with the “~” character play a special role. For example, the `~fuzzysets` field works as a map $f_{sn} \rightarrow md$, where f_{sn} is a fuzzy-set name and md is the corresponding membership degree (see Fig. 4a); this way, the degrees of membership to multiple fuzzy sets of a document can be simultaneously represented. The model encompasses the `~geometry` field too, for representing spatial geometries [2,15]; however, this is outside the scope of this paper.
- *Declarative Instructions.* *J-CO-QL* provides declarative instructions, suitable for specifying complex transformations of *JSON* data sets. Furthermore, instructions handle documents with different structures at the same time.
- *Execution Model.* A clear and linear execution model has been defined. The query (or script) is a sequence of instructions. The *query-process state* $\mathbf{s}$ contains the so-called *temporary collection* $\mathbf{s.tc}$: an instruction $\mathbf{i}_j$ works on the $\mathbf{s}_{(j-1)}$ query-process state and generates a new query-process state $\mathbf{s}_j$, meaning that $\mathbf{i}_j$ receives the $\mathbf{s}_{(j-1).tc}$ temporary collection as input and generates the new $\mathbf{s}_j.tc$ temporary collection as output; in the initial state $\mathbf{s}_0$, it is $\mathbf{s}_0.tc = \emptyset$. Consequently, instructions are *pipelined*, so as the user can write them following the natural flow with which they are thought.

The *J-CO* Framework is composed by several software tools, as depicted in Fig. 1. We report a brief summary of them (the interested reader can refer to [15]). The *J-CO-QL Engine* actually processes *J-CO-QL* scripts. It is able to access Web sources and can connect to *JSON* stores like *MongoDB*. It can connect to *J-CO-DS* as well: it is a *JSON* document store developed within the *J-CO* Framework, which can store very-large single *JSON* documents into its databases; it does not provide any processing capability (no query language is provided). Finally, *J-CO-UI* is the user interface of the framework: users can use it to interact with the *J-CO-QL Engine*.

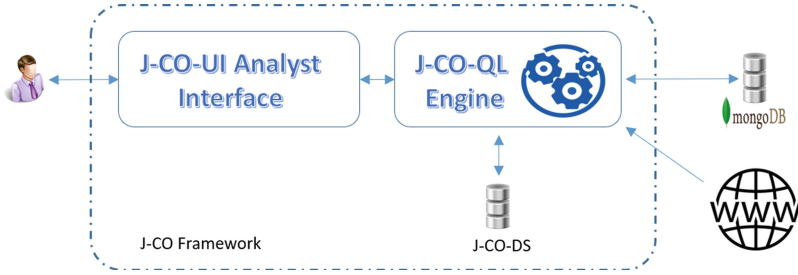


Fig. 1. Components of the *J-CO* Framework

3 Case Study and *J-CO-QL* Script

We now address the main contribution of the paper: we show how *J-CO-QL* can integrate JavaScript functions to perform soft queries on *JSON* data sets.

To this end, we defined a case study that is inspired by real data sets available on the Open Data portal of *Regione Lombardia*¹, the administrative body of the region named Lombardia in northern Italy. This portal is based on *Socrata*², one of the *de-facto* standard platforms for building Open Data portals, which provides data by means of web APIs.

Hereafter, we define the problem addressed by the case study.

Problem 1. *Given the pool of air-quality sensors in the area of Regione Lombardia, find the pairs of a sensor for nitric oxide and of a sensor for $PM_{2.5}$ that are closely located (but not in the same neighborhood) in the same municipality.*

To address Problem 1, we wrote a *J-CO-QL* script that is divided in three different parts, each one devoted to perform a specific sub-task.

- **Part 1** (reported in Listing 1 and explained in Sect. 3.1) is in charge (i) for defining a JavaScript function that calculates the geodetic distance of two points on the Earth globe, and (ii) for defining a specific Fuzzy Operator that exploits the JavaScript function that enables the soft query performed by Part 3 (reported in Listing 3).
- **Part 2** (reported in Listing 2 and explained in Sect. 3.2) (i) retrieves data from Regione Lombardia Open Data portal and then (ii) pre-processes the data sets.
- **Part 3** (reported in Listing 3 and explained in Sect. 3.3) (i) performs the soft query on the pre-processed data through the Fuzzy Operator defined in Part 1, and (ii) saves the results into a *MongoDB* database.

¹ Regione Lombardia Open Data Portal - <https://www.dati.lombardia.it/>.

² Socrata platform - <https://dev.socrata.com/>.

Listing 1 *J-CO-QL* script, part 1: Definitions.

```

1. CREATE JAVASCRIPT FUNCTION GeoKmDistance
  PARAMETERS
    lat1 TYPE Float, lon1 TYPE Float, lat2 TYPE Float, lon2 TYPE Float
  PRECONDITION
    lat1 >= -90    AND lat1 <= +90    AND lat2 >= -90    AND lat2 <= +90    AND
    lon1 >= -180   AND lon1 <= +180   AND lon2 >= -180   AND lon2 <= +180
  BODY {
    var R = 6371;           // Earth radius in km
    var dLat = (lat2-lat1) * (Math.PI/180);
    var dLon = (lon2-lon1) * (Math.PI/180);
    var a = Math.sin(dLat/2) * Math.sin(dLat/2) +
            Math.cos(lat1 * (Math.PI/180)) *
            Math.cos(lat2 * (Math.PI/180)) *
            Math.sin(dLon/2) * Math.sin(dLon/2) ;
    var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
    var d = R * c;          // Distance in km
    return d;
  } END BODY ;

2. CREATE FUZZY OPERATOR Close
  PARAMETERS
    lat1 TYPE Float, lon1 TYPE Float, lat2 TYPE Float, lon2 TYPE Float
  EVALUATE GeoKmDistance(lat1, lon1, lat2, lon2)
  POLYLINE
    [ ( 0.0, 0.00),
      ( 0.5, 0.00),
      ( 0.6, 1.00),
      ( 1.0, 1.00),
      ( 3.0, 0.90),
      ( 6.0, 0.30),
      (10.0, 0.10),
      (15.0, 0.00),
      (20.0, 0.00) ] ;

```

3.1 JavaScript Function and Fuzzy Operator

The *J-CO-QL* sub-script for Part 1 is reported in Listing 1. Its goal is (i) defining a JavaScript function that calculates the geodetic distance of two points (given their coordinates), and (ii) defining a Fuzzy Operator that exploits the JavaScript function to evaluate the “degree of closeness” of two points. We now present each single instruction in details, by explaining its semantics.

The **CREATE JAVASCRIPT FUNCTION** instruction on line 1 defines the JavaScript function called **GeoKmDistance**. We describe it in details.

- The **PARAMETERS** clause declares the formal parameters. In this case, **lat1** and **lon1** represent, respectively, the latitude and longitude of the first point, while **lat2** and **lon2** represent, respectively, the latitude and longitude of the second point. All parameters are floating-point numbers.
- The **PRECONDITION** clause specifies a condition that must be met by actual parameters to execute the JavaScript function. In this case, latitudes must be within the $[-90^\circ, +90^\circ]$ range, while longitudes must be within the $[-180^\circ, +180^\circ]$ range.
- The following **BODY** section contains the actual JavaScript code, which in our case calculates the geodetic distance (in *Km*) by means of the coordinates

received as actual values for the parameters defined in the **PARAMETERS** clause, by applying the *Haversine formula*³.

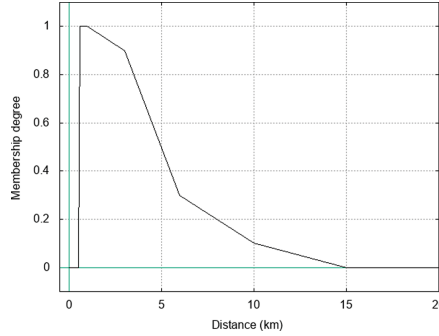


Fig. 2. Membership function of the **Close** fuzzy operator.

The **CREATE FUZZY OPERATOR** instruction on line 2 in Listing 1 defines a fuzzy operator, named **Close**, that evaluates the degree of closeness of two points.

- The **PARAMETERS** clause declares the parameters that will be used by the fuzzy operator. In this case, **lat1** and **lon1** represent, respectively, the latitude and longitude of the first point, while **lat2** and **lon2** represent, respectively, the latitude and longitude of the second point. All parameters are floating-point numbers.
- The **EVALUATE** clause defines the mathematical function whose resulting value is used to get the actual membership degree. The evaluation expression calls the **GeoKmDistance** JavaScript function defined on line 1 to obtain the distance (in *Km*) between two different points.
- The **POLYLINE** clause defines the *membership function* of the fuzzy operator as a polyline. It is described by a list of $n + 1$ points (x_i, y_i) , with $0 \leq i \leq n$, $x_i < x_{i+1}$ and $0 \leq y_i \leq 1$. When $x < x_0$, the corresponding y -value is assumed to be equal to y_0 ; when $x > x_n$, the corresponding y -value is assumed to be equal to y_n . Figure 2 depicts the defined polyline.

The value returned by the expression in the **EVALUATE** clause is used as x -value for the *membership function*, which provides the corresponding y -value as membership degree.

Figure 2 depicts the membership function we defined for the **Close** fuzzy operator. The rationale for the particular *descending S-shape* curve is that when two sensors are too close each other they are in the same neighborhood (Problem 1 excludes them). This way, the membership degree is 0 up to 0.5 *Km*, so as

³ *Haversine formula* to calculate distance between two geo-referenced points https://en.wikipedia.org/wiki/Haversine_formula.

very close sensors are not considered as “close”. Notice the complex shape of the curve, which is made possible by the polyline; this way, the user can calibrate the semantics of the “close” relationship expressed by the fuzzy operator.

Notice how the semantics of a fuzzy operator is quite rich by itself. However, the possibility to call JavaScript functions in the **EVALUATE** expression makes it possible to define fuzzy operators that rely on very complex semantics, where the JavaScript function performs complex (imperative/procedural) computations on parameters. In the case study, the **GeoKmDistance** function computes the distance in a procedural way; as a result, the **EVALUATE** expression in the fuzzy operator becomes a simple call to the function, but the semantics of the operator relies on the complex (or, at least, non-trivial) computation of the geodetic distance. The overall semantics of the fuzzy operator is significantly powered by the introduction of JavaScript functions in the language.

3.2 Retrieving and Preprocessing Data

Listing 2 reports the *J-CO-QL* sub-script for Part 2. It has to: (i) retrieve data about air-quality sensors located in Lombardia from Regione Lombardia Open Data portal; (ii) transform and format these data in a convenient way; (iii) create a collection of documents that describe pairs made by a sensor for *nitric oxide* and a sensor for *particulate matter* ($PM_{2.5}$) located in the same municipality.

The **GET COLLECTION** instruction on line 3 retrieves data about the air-quality sensors from Regione Lombardia Open Data portal, by sending the request to the dedicated end point⁴. This is done by specifying the URL which to send the HTTP request. In the URL, notice the query string after the question mark: it is written according to the query language accepted by *Socrata* (that powers the portal); the query specifies the maximum number of documents to return. The data are provided as a unique array of *JSON* documents, one for each sensor; there are different kinds of sensor, each one performing a different measurement type, such as (i) level of *nitric oxide*, (ii) level of *ozone*, (iii) level of *carbon monoxide*, and so on. The array becomes the value of the lonely **data** field in the unique document within the new temporary collection: Fig. 3a shows an excerpt with the first two documents in the array. Notice that most of the field names are in Italian and that all their values are reported as string values.

The **EXPAND** instruction on line 4 unnests documents within the **data** field, to obtain one document for each sensor. We present the instruction in details.

- The **UNPACK** clause selects, by means of the **WITH** predicate, the lonely document in the input temporary collection, since it has the **data** array field.
- The **ARRAY** sub-clause specifies the field to expand (the **data** field) and the **TO** keyword precedes the name of the field to generate into the new documents by unnesting an item in the source array: given a document to expand,

⁴ Regione Lombardia Open Data Portal. Air-quality stations - <https://www.dati.lombardia.it/Ambiente/Stazioni-qualit-dell-aria/ib47-atvt>.

Listing 2 *J-CO-QL* script, part 2: Data retrieval and preprocessing.

```

3. GET COLLECTION FROM WEB
   "https://www.dati.lombardia.it/resource/ib47-atvt.json?$limit=10000";

4. EXPAND
   UNPACK WITH ARRAY .data
     ARRAY .data TO .sensor
   DROP OTHERS;

5. FILTER
   CASE
     WHERE WITH .sensor.item.idsensore, .sensor.item.comune,
              .sensor.item.nometiposensore
           AND .sensor.item.nometiposensore = "Ossidi di Azoto"
     GENERATE { .city      : .sensor.item.comune,
                 .latitude : TO_FLOAT (.sensor.item.lat),
                 .longitude : TO_FLOAT (.sensor.item.lng),
                 .sensorId  : .sensor.item.idsensore,
                 .sensorType : "Nitric Oxide" }
     WHERE WITH .sensor.item.idsensore, .sensor.item.comune,
              .sensor.item.nometiposensore
           AND .sensor.item.nometiposensore = "Particelle sospese PM2.5"
     GENERATE { .city      : .sensor.item.comune,
                 .latitude : TO_FLOAT (.sensor.item.lat),
                 .longitude : TO_FLOAT (.sensor.item.lng),
                 .sensorId  : .sensor.item.idsensore,
                 .sensorType : "PM 2.5" }
   DROP OTHERS;

6. JOIN OF COLLECTIONS temporary AS t1, temporary AS t2
   CASE
     WHERE .t1.city = .t2.city
           AND .t1.sensorType = "Nitric Oxide"
           AND .t2.sensorType = "PM 2.5"
     GENERATE { .city      : .t1.city,
                 .N0sensorId : .t1.sensorId,
                 .N0latitude : .t1.latitude,
                 .N0longitude : .t1.longitude,
                 .PM25sensorId : .t2.sensorId,
                 .PM25latitude : .t2.latitude,
                 .PM25longitude : .t2.longitude }
   DROP OTHERS;

```

for each item in the array a novel document is generated; all fields, in the source document, are kept, except for the expanded array, while a new field is added. In this case, since the input document has only the **data** array field (to expand), the output documents have only the **sensor** field.

- Finally, the **DROP OTHERS** option discards all those documents in the input temporary collection that are not selected by the **UNPACK** clause; clearly, in this case it is ineffective.

Figure 3b shows an example of the 959 documents generated by the instruction: notice that the **sensor** field is a structured field with the **item** field, which contains the unnested document, and the **position** field, which denotes the position of the unnested item in the source array.

The **FILTER** instruction on line 5 filters and transforms documents in the temporary collection generated by line 4. In the **CASE** clause, there are two distinct **WHERE** conditions: for each document d in the input temporary collection, (i) if d matches the condition expressed in the first **WHERE** condition, then (ii) the following **GENERATE** action is applied to d and the subsequent **WHERE** condition(s) is(are) ignored; (iii) if d does not match the first **WHERE** condition, then

```
{ "data" : [
  { "comune"      : "Cassano d'Adda",
    "idsensore"   : "10001",
    "lat"         : "45.52647254585104",
    "lng"         : "9.515980483627096",
    "nometiposensore" : "Ossidi di Azoto",
    ... uninteresting other fields ... },
  { "comune"      : "Cassano d'Adda",
    "idsensore"   : "10002",
    "lat"         : "45.52647254585104",
    "lng"         : "9.515980483627096",
    "nometiposensore" : "Monossido di Carbonio",
    ... uninteresting other fields ... },
  ... ]
```

(a) Excerpt of document after line 3.

```
{ "sensor" : {
  "item" : {
    "comune"      : "Truccazzano",
    "idsensore"   : "10004",
    "lat"         : "45.48455813667995",
    "lng"         : "9.471348388974729",
    "nometiposensore" : "Ossidi di Azoto",
    ... uninteresting other fields ... },
  "position" : 2 } }
```

(b) Example of document after line 4.

```
{ "city"      : "Truccazzano",
  "latitude"  : 45.48455813668,
  "longitude" : 9.47134838897473,
  "sensorId"  : "10004",
  "sensorType" : "Nitric Oxide" }
```

(c) Example of document after line 5.

```
{ "NOLatitude" : 45.1425406322169,
  "NOLongitude" : 10.0438361583694,
  "NOSensorId" : "9930",
  "PM25Latitude" : 45.1319469128517,
  "PM25Longitude" : 10.015741513336,
  "PM25SensorId" : "12577",
  "city" : "Cremona" }
```

(d) Example of document after line 6.

Fig. 3. Sample documents in the temporary collection generated by line 3 (a), by line 4 (b), by line 5 (c) and by line 6 (d).

(iv) the second **WHERE** condition is evaluated on d and if d matches it the following **GENERATE** action is applied, otherwise (v) d is discarded, according to the final **DROP OTHERS** option (multiple **WHERE** conditions enable the instruction to manage heterogeneous documents). Specifically:

- The first **WHERE** condition selects those documents that have the fields that are listed in the **WITH** predicate, and whose values identify a *nitric oxide* sensor (`.sensor.item.tipologia = "Ossidi di Azoto"`). The **GENERATE** action generates a new output document in place of the selected one; a field value can be either (i) a constant value (e.g., `.sensorType : "Nitric Oxide"`), or (ii) a value taken as is from a source field (e.g., `.sensorId : .sensor.item.idsensore`), or (iii) a value obtained by converting the value of a source field from one data type to another (e.g., `.latitude:TO_FLOAT (.sensor.item.lat)`, which converts a string into a floating-point number); the remaining source fields are discarded.
- The second **WHERE** condition and **GENERATE** action work similarly to the previous ones; they select and rebuild only those documents that describe $PM_{2.5}$ sensors (`.sensor.item.tipologia = "Particelle sospese PM2.5"`).

Figure 3c shows an example of the 205 documents selected by the instruction. Notice that now all the fields have an English name and their values are numerical if they represent a numerical property (e.g., the `latitude` field).

On line 6, there is a **JOIN** instruction. Given two collections named **L** (for left) and **R** (for right), the **JOIN** instruction pairs each document $l \in L$ to each document $r \in R$: for each pair, a document d is built, with two distinct structured fields **L** and **R** containing, respectively, the l document and the r document.

Listing 3 *J-CO-QL* script, part 3: Soft querying.

```

7.  FILTER
    CASE
      WHERE WITH .Nolatitude, .Nolongitude, .PM25latitude, .PM25longitude
      CHECK FOR FUZZY SET CloseSensors
      USING Close (.Nolatitude, .Nolongitude, .PM25latitude, .PM25longitude)
      ALPHA-CUT 0.9 ON CloseSensors
      DROP OTHERS;

8.  FILTER
    CASE
      WHERE KNOWN FUZZY SETS CloseSensors
      GENERATE { .city,
                  .NosensorId,
                  .PM25sensorId,
                  .rank
                : MEMBERSHIP_OF (CloseSensors)  }
      DROPPING ALL FUZZY SETS
      DROP OTHERS;

9.  USE DB SOCO_2021 ON SERVER MongoDB 'http://127.0.0.1:27017';

10. SAVE AS CloseSensors@SOCO_2021;

```

- Specifically, the instruction joins the temporary collection created by line 5 with itself, by using the two different aliases **t1** and **t2**; each generated document contains the **t1** and **t2** fields, so as to describe a couple of sensors.
- The **WHERE** condition selects only those documents that describe two sensors that are in the same municipality (**.t1.city = .t2.city**), such that the *left* sensor measures *nitric oxide*, while the *right* sensor measures *PM_{2.5}*.
- The subsequent **GENERATE** action flattens the document, by reporting fields only describing the city, the ID and the coordinates of the two sensors.
- The final **DROP OTHERS** option discards all documents that do not meet the **WHERE** condition from the output collection.

Figure 3d shows an example of the 101 documents generated by the instruction.

3.3 Soft Querying

Listing 3 reports the *J-CO-QL* sub-script for Part 3. It has to find documents in the temporary collection created by line 6 such that they describe a pair of sensors that are close but not in the same neighborhood. The degree of closeness is evaluated through the **Close** fuzzy operator, in a soft way. Documents that report a sufficiently-high degree of closeness are saved into a *MongoDB* database.

Compared to line 5, the **FILTER** instruction on line 7 shows novel clauses.

- The **WHERE** condition selects only those documents with latitude and longitude coordinates of the paired sensors. Notice that, by construction, all documents in the input temporary collection satisfy these requirements.
- For each document selected by the **WHERE** condition, the subsequent **CHECK FOR FUZZY SET** clause evaluates the membership degree to a fuzzy set named **CloseSensors**; the evaluation is done by the **USING** fuzzy condition, that exploits the **Close** fuzzy operator (defined on line 2) to evaluate the degree of closeness based on the geodetic distance between sensors (by means of their coordinates as actual parameters).

```
{ "N0latitude"      : 45.1425406322169,
  "N0longitude"     : 10.0438361583694,
  "N0sensorId"      : "9930",
  "PM25latitude"    : 45.1319469128517,
  "PM25longitude"   : 10.015741513336,
  "PM25sensorId"    : "12577",
  "city"            : "Cremona",
  "~fuzzysets"      : {
    "CloseSensors"  : 0.925061366449008 } }
```

(a) Example of document after line 7.

```
{ "N0sensorId"      : "9930",
  "PM25sensorId"    : "12577",
  "city"            : "Cremona",
  "rank"            : 0.925061366449008 }
```

(b) Example of document after line 8.

Fig. 4. Sample documents in the temporary collection generated by line 7 (a) and by line 8 (b).

- The **ALPHA-CUT** action discards all documents whose degree of membership to the **CloseSensors** fuzzy set is less than the threshold of 0.9, keeping only those documents that denote a very-strong relationship of closeness between two sensors.
- The final **DROP OTHERS** option, by construction, is ineffective.

Figure 4a shows an example of the 27 documents generated by the **FILTER** instruction on line 7. Notice the root-level **~fuzzysets** field, reporting the degree of membership of the document to the **CloseSensors** fuzzy set.

Observe that, in the case that coordinates of sensors are wrong, the pre-condition of the **GeoKmDistance** JavaScript function (line 1 of Listing 1) is not met; this causes the raising of an error status, that propagates back through the **Close** fuzzy operator (that called the function); the evaluation of the fuzzy set is stopped and, eventually, the document is discarded by the **ALPHA-CUT** action.

The **FILTER** instruction on line 8 “de-fuzzifies” documents created by line 7.

- The **WHERE** condition, by means of the **KNOWN FUZZY SETS** predicate, selects documents for which the **CloseSensors** fuzzy set has been evaluated (in this specific case, by construction, all documents are selected).
- The **GENERATE** action transforms all documents reporting only the city, and the IDs of the sensors. In addition, a new **rank** field is created, which reports the membership degree to the **CloseSensors** fuzzy set; this field tells the user how close the sensors are. The **DROPPING ALL FUZZY SETS** option discards the special **~fuzzysets** field from the documents (i.e., it “de-fuzzifies” them).
- The final **DROP OTHERS** option, by construction, is ineffective.

Figure 4b shows an example of the 27 final documents generated by line 8.

On line 9, the **USE DB** instruction asks to connect to the **SOCO.2021** database managed by the *MongoDB* server installed on the computer where the *J-CO-QL Engine* is running.

Finally, the **SAVE AS** instruction on line 10 saves persistently the current temporary collection into the *MongoDB* database with name **CloseSensors**.

Discussion. We want to conclude this section with a few remarks. The reader can notice that by evaluating the extent to which a document belongs to a fuzzy set enables partial matching of documents. The sample document in Fig. 4a

describes two sensors that are not fully close, while they are almost close with an extent of 92.5 %. This way, documents that do not completely match user wishes, but are close to them, are retrieved. Furthermore, the membership degree can be used by users to rank output documents, based on their relevance in relation to the query.

Again, we want to stress the contribution of JavaScript functions to define fuzzy operators with complex semantics. This is a little but fundamental brick to make *J-CO-QL* able to provide soft-querying capabilities on any kind of *JSON* document. Currently, only the `FILTER` instruction has been extended to support fuzzy set evaluation on *JSON* documents, but we are going to extend the other instructions provided by *J-CO-QL*.

4 Conclusions

In this paper, we presented a short case study by means of which we show soft-querying capabilities provided by *J-CO-QL* on collections of *JSON* documents. *J-CO-QL* is the query language of the *J-CO* Framework, under development at University of Bergamo (Italy) to address the problem of retrieving, managing, integrating and querying *JSON* data sets available on Web sources or within *NoSQL* data stores. *J-CO-QL* is under continuous evolution and, currently, we are primarily extending it with soft-querying capabilities. The novel contribution of this paper (with respect to past works [10,15,16] is the introduction of the `CREATE JAVASCRIPT FUNCTION` instruction: this instruction allows users to define JavaScript functions that can be called by expressions in the language. This functionality makes it possible to use JavaScript functions to define complex semantics for fuzzy operators, which are the key for performing soft queries.

Thus, the main contribution of the paper is to show how JavaScript functions can power the definition of fuzzy operators, contributing to define complex semantics. We rely on a case study that moves from real data sets available on an Open Data portal. The *J-CO-QL* script briefly shows the capabilities of the language, which is general and effective.

As a future work, the first direction we are going to follow is the extension of other instructions in the language with fuzzy set evaluation, such as the `JOIN` instruction. In fact, joining two documents having both some evaluated fuzzy sets is not a trivial task; thus, the fuzzy extension of the `JOIN` instruction must be carefully designed.

Furthermore, we are going to design and develop novel tools to include within the *J-CO* Framework; in particular, we are thinking to a novel user interface, which should provide users with active support to inspect data and suggest how to write *J-CO-QL* instructions.

The *J-CO* Framework is available on a public GitHub repository⁵.

⁵ Github repository of the *J-CO* Framework - <https://github.com/zunstraal/J-Co-Project>.

References

1. Blair, D.C.: Information retrieval, 2nd ed. c.j. van rijnsbergen. london: Butterworths; 1979: 208 pp. price: \$32.50. J. Am. Soc. Inf. Sci. **30**(6), 374–375 (1979). <https://doi.org/10.1002/asi.4630300621>
2. Bordogna, G., Capelli, S., Ciriello, D.E., Psaila, G.: A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: the case study of volunteered personal traces analysis against transport network data. Geo-spatial Inf. Sci. **21**(3), 257–271 (2018)
3. Bordogna, G., Capelli, S., Psaila, G.: A big geo data query framework to correlate open data with social network geotagged posts. In: The Annual International Conference on Geographic Information Science, pp. 185–203. Springer, Heidelberg (2017). https://doi.org/10.1007/978-3-319-56759-4_11
4. Bordogna, G., Psaila, G.: Soft aggregation in flexible databases querying based on the vector p-norm. Int. J. Uncertainty Fuzz. Knowl.-Based Syst. **17**(supp01), 25–40 (2009)
5. Bordogna, G., Psaila, G.: Customizable flexible querying in classical relational databases. In: Handbook of Research on Fuzzy Information Processing in Databases, pp. 191–217. IGI Global (2008)
6. Bosc, P., Prade, H.: An introduction to the fuzzy set and possibility theory-based treatment of flexible queries and uncertain or imprecise databases. In: Uncertainty Management in Information Systems, pp. 285–324. Springer, Heidelberg (1997). https://doi.org/10.1007/978-1-4615-6245-0_10
7. Bosc, P., Pivert, O.: Sqlf: a relational database language for fuzzy querying. IEEE Trans. Fuzzy Syst. **3**(1), 1–17 (1995)
8. Bray, T.: The javascript object notation (json) data interchange format (2014). <https://www.rfc-editor.org/rfc/rfc7159.txt>
9. Colpaert, P., Joye, S., Mechant, P., Mannens, E., Van de Walle, R.: The 5 stars of open data portals. In: Proceedings of the 7th International Conference on Methodologies, Technologies and Tools Enabling E-Government (MeTTeG13), University of Vigo, Spain, pp. 61–67 (2013)
10. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of json data sets through fuzzy sets: a case study. Information **12**(7), 258 (2021)
11. Kacprzyk, J., Zadrozny, S.: Fquery for access: fuzzy querying for a windows-based dbms. In: Bosc, P., Kacprzyk, J. (eds), Fuzziness in Database Management Systems. Studies in Fuzziness, vol. 5 (1995)
12. Kraft, D.H., Petry, F.E.: Fuzzy information systems: managing uncertainty in databases and information retrieval systems. Fuzzy Sets Syst. **90**(2), 183–191 (1997)
13. Medina, J.M., Pons, O., Vila, M.A.: Gefred: a generalized model of fuzzy relational databases. Inf. Sci. **76**(1), 87–109 (1994)
14. Psaila, G., Fosci, P.: Toward an analyst-oriented polystore framework for processing json geo-data. In: International Conferences on Applied Computing 2018, Budapest, Hungary, 21–23 October 2018, pp. 213–222. IADIS (2018)
15. Psaila, G., Fosci, P.: J-co: a platform-independent framework for managing geo-referenced json data sets. Electronics **10**(5), 621 (2021)

16. Psaila, G., Marrara, S.: A first step towards a fuzzy framework for analyzing collections of json documents. IADIS AC **2019**, 19–28 (2019)
17. Zadeh, L.A.: The concept of a linguistic variable and its application to approximate reasoning–i. Inf. Sci. **8**(3), 199–249 (1975)